

<CodeLab>

VOXXEDDAYS — AMSTERDAM



koturk77



ko-turk



koturk.nl



koturk.nl



ABOUT KO TURK

Developer, architect, conference organiser



JUGLeader @CodeLabJUG



Speaker



***MAVEN'S HIDDEN SECRETS AND TREASURES TO SPEED
UP YOUR BUILD***



GOLDEN AGE OF PIRACY





BLACKBEARD

Born	Edward Teach c. 1680 (presumed) Bristol, England
Died	22 November 1718 (aged 35–40) Ocracoke, North Carolina, British America
Cause of death	Killed in action (stab wounds & gunshot wounds)
	Piratical career



OUR JOURNEY

TREASURE MAP

A treasure map with a red dashed line path. The path starts at the bottom left, goes up to the left, then curves around the top and right, and finally goes down to the bottom right. The path is marked with a red 'X' at the top. The map features a compass rose in the top left, a skull in the top right, and a ship in the bottom left. The background is a detailed landscape with mountains, a sea monster, and a treasure chest.

CACHING

PARALLEL

PROFILES

SCOPES

ANNOTATIONS

ANALYZE

PITFALLS

SETUP MAVEN

LIFECYCLES

PROFILING

DAEMON

TREASUREMAP

X
CACHING

PARALLEL

PROFILES

SCOPES

ANNOTATIONS

ANALYZE

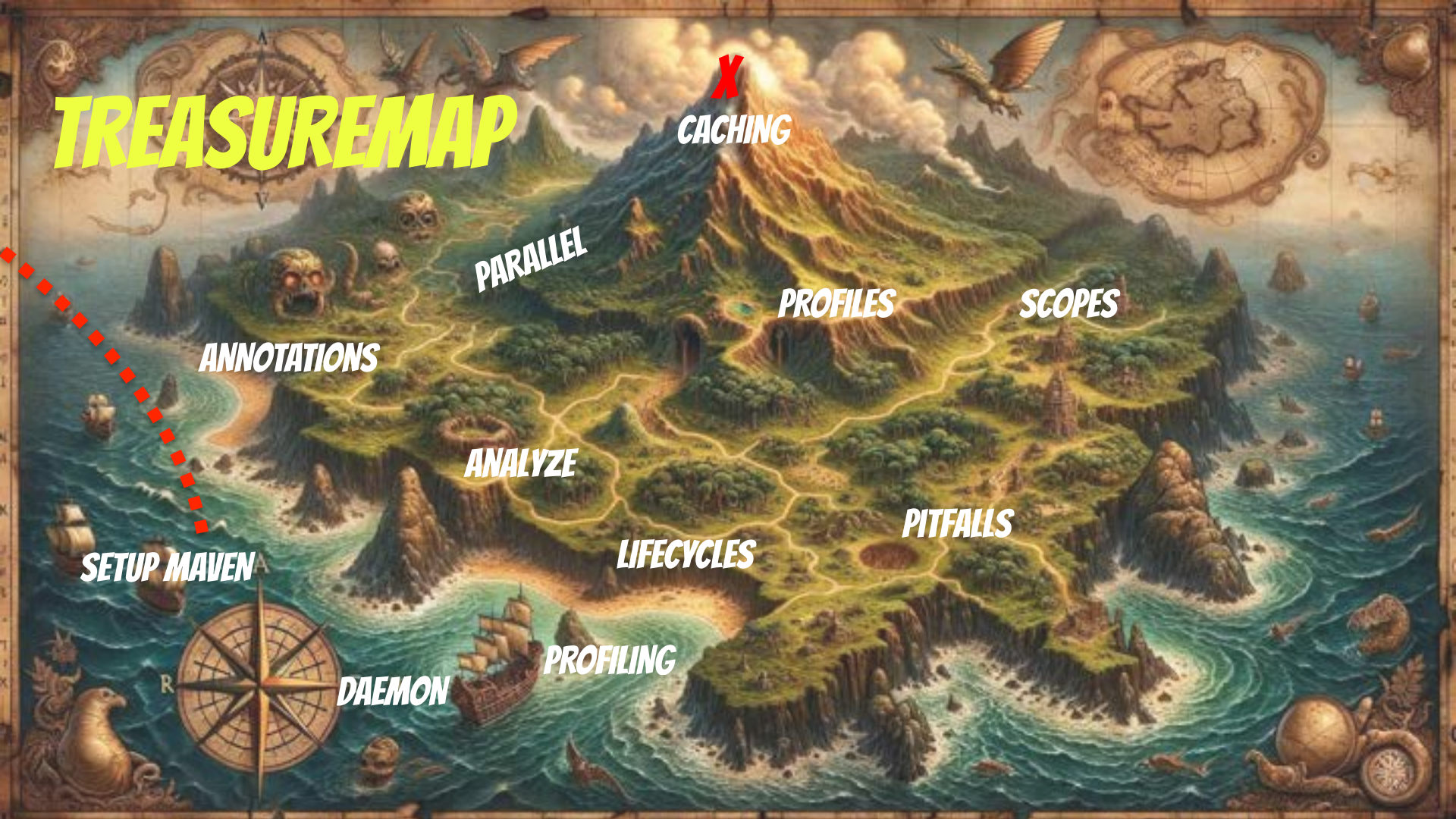
PITFALLS

LIFECYCLES

SETUP MAVEN

PROFILING

DAEMON





WHO IS USING

MAVEN? GRADLE? ANT?

BUILDING WITH MAVEN

2025: ??? Let's see the survey.

2024: Maven 73 %, Gradle 46 %, Ant 5–6%

2023: Maven 74 %, Gradle 46 %, Ant 6 %

2022: Maven 73 %, Gradle 50 %, Ant 8 %

2021: Maven 72 %, Gradle 49 %, Ant 8 %

2020: Maven 71 %, Gradle 48 %, Ant 9 %

2019: Maven 70 %, Gradle 53 %, Ant 13 %

2018: Maven 68 %, Gradle 47 %, Ant 11 %

2017: Maven 71 %, Gradle 42 %, Ant 15 %

**Based on the State of Developer EcoSystem
Jetbrains**

HOW IT STARTED



Maven was created by Jason van Zyl in 2002 and began as a sub-project of [Apache Turbine](#).

In 2003 Maven was accepted as a top level [Apache Software Foundation](#) project.

CURRENT STATUS

Maven 3.9.12

- production / stable

Maven 4.0-rc5

- not yet for production

CURRENT STATUS MAVEN 4 (RC-5)

- maven knows it's context
 - less complexity (fields in children can be deleted)
 - parallel building is better
 - better error handling
-
- mvnup check -> for migrating
 - Java 17

OPTIONAL: DOWNLOAD MAVEN

DOWNLOAD VIA SITE: [***HTTPS://MAVEN.APACHE.ORG/DOWNLOAD.CGI***](https://maven.apache.org/download.cgi)

DOWNLOAD VIA BREW:

```
$ brew install maven
```

DOWNLOAD VIA SDK:

```
$ sdk install maven <version>
```

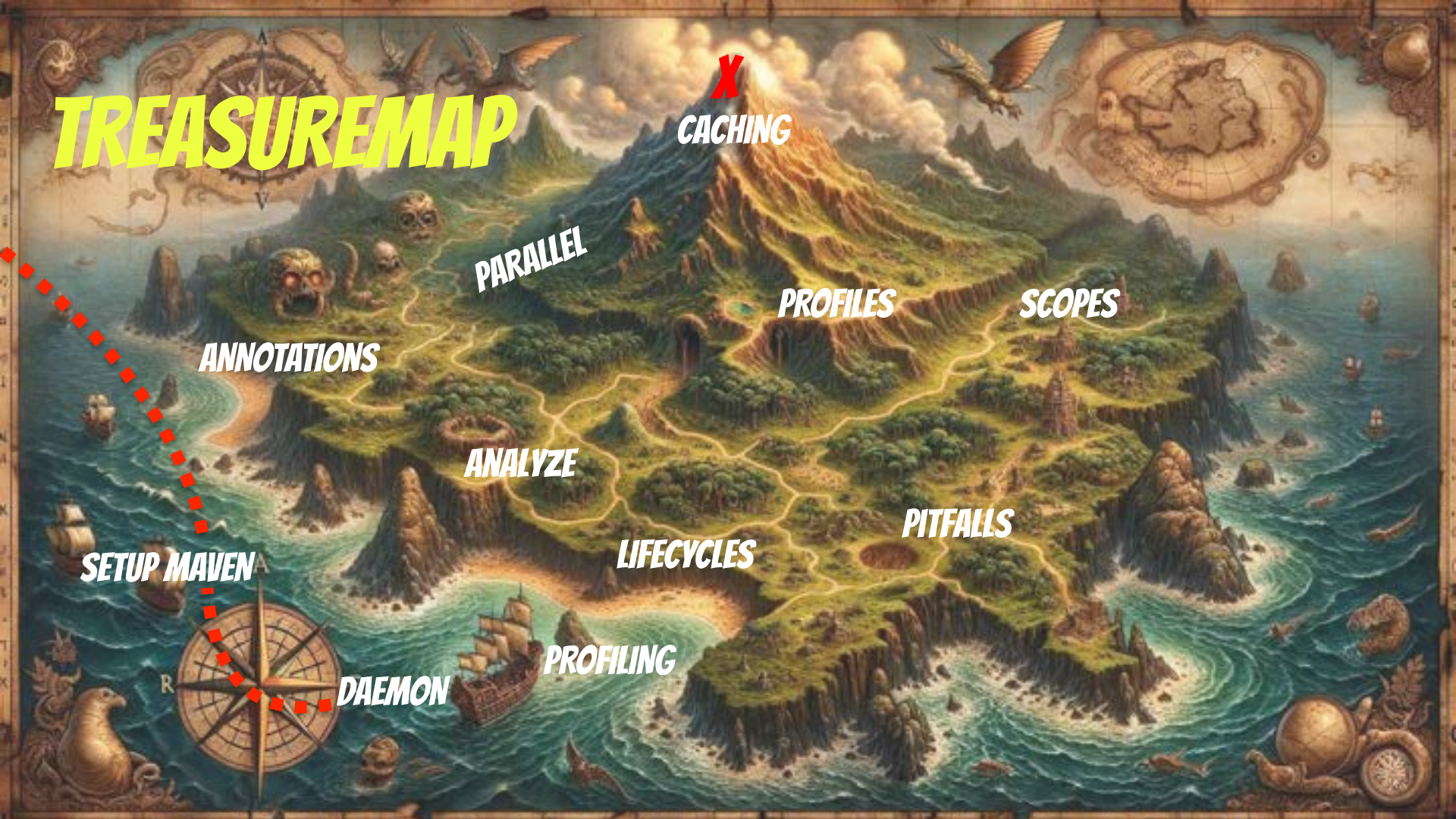
SPK MAN!

OR USE A MVN WRAPPER

```
$ mvn -N wrapper:wrapper -Dmaven=3.9.12
```



TREASUREMAP



A detailed, ornate treasure map of a fantastical island. The map features a central mountain range, a sea monster on the left, and a treasure chest on the right. The map is labeled with various terms related to software development and testing. A red dashed line runs diagonally across the map, starting from the top left and ending near the bottom left. The labels are as follows:

- TREASUREMAP** (Large yellow text at the top left)
- CACHING** (Red 'X' above the word, top center)
- PARALLEL** (Top center, below CACHING)
- PROFILES** (Top right, below PARALLEL)
- SCOPES** (Top right, below PROFILES)
- ANNOTATIONS** (Middle left, below PARALLEL)
- ANALYZE** (Middle left, below ANNOTATIONS)
- LIFECYCLES** (Middle right, below ANALYZE)
- PITFALLS** (Middle right, below LIFECYCLES)
- SETUP MAVEN** (Bottom left, below ANNOTATIONS)
- DAEMON** (Bottom left, below SETUP MAVEN)
- PROFILING** (Bottom center, below LIFECYCLES)

X
CACHING

PARALLEL

PROFILES

SCOPES

ANNOTATIONS

ANALYZE

LIFECYCLES

PITFALLS

SETUP MAVEN

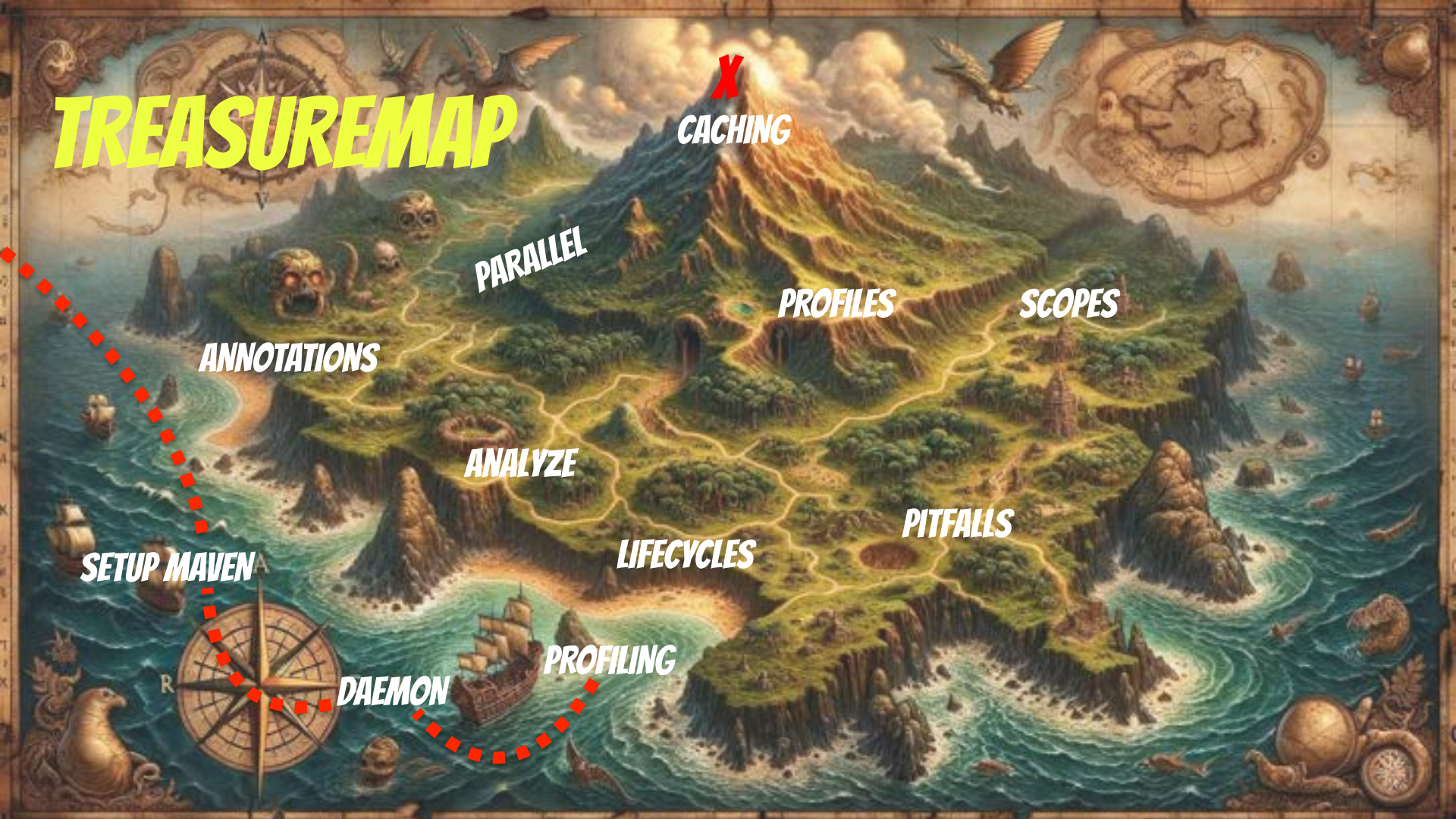
DAEMON

PROFILING

DAEMON

IS IT FASTER? LET'S TEST

TREASUREMAP



X
CACHING

PARALLEL

PROFILES

SCOPES

ANNOTATIONS

ANALYZE

PITFALLS

LIFECYCLES

PROFILING

DAEMON

SETUP MAVEN

PROFILING

YOUR BUILD

PROFILING WITH JCGAY/MAVEN-PROFILER

Adding .mvn/extensions.xml file:

```
<?xml version="1.0" encoding="UTF-8"?>
<extensions>
  <extension>
    <groupId>fr.jcgay.maven</groupId>
    <artifactId>maven-profiler</artifactId>
    <version>3.3</version>
  </extension>
</extensions>
```

```
mvn clean install -Dprofile
```

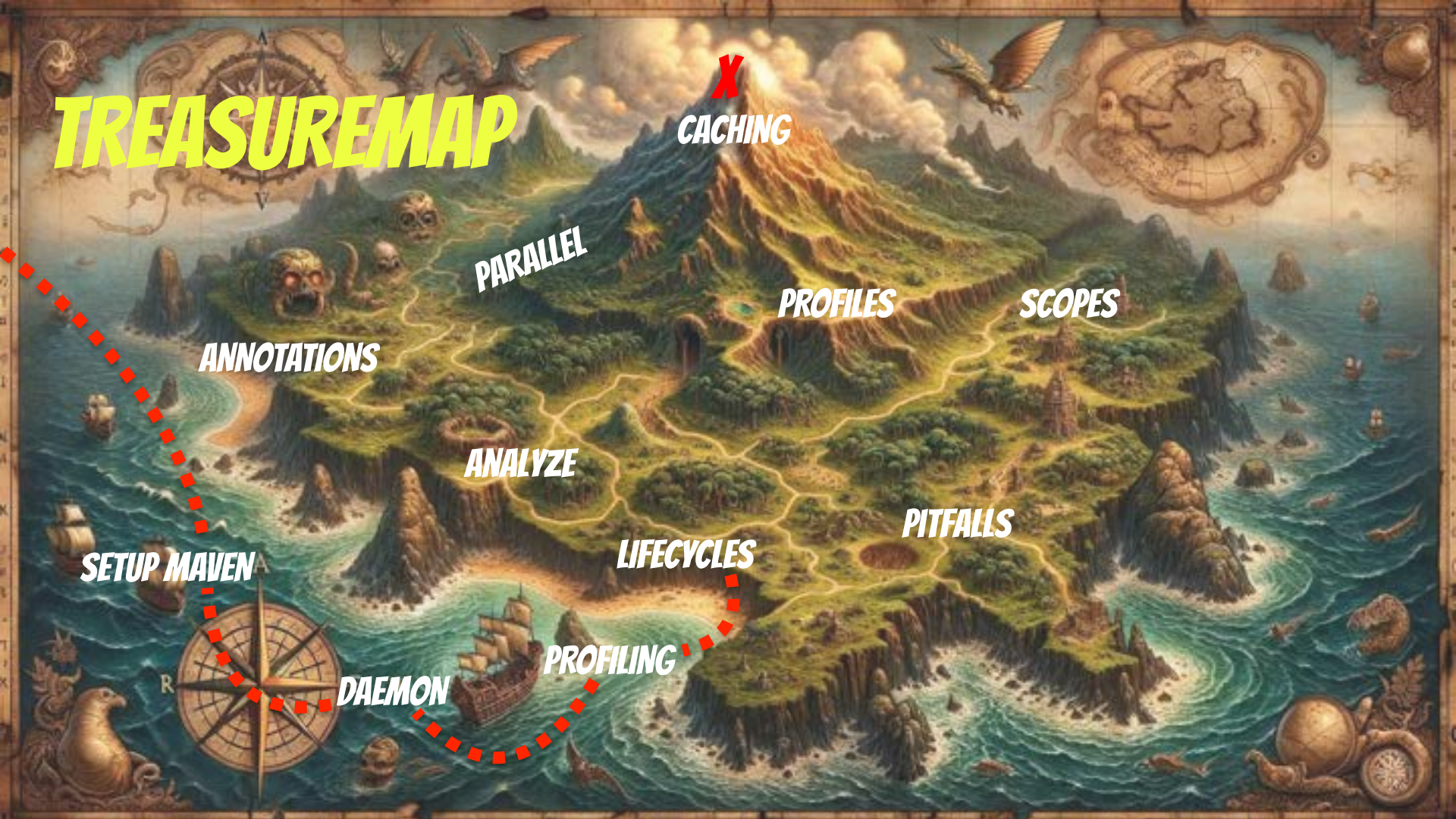
PROFILING

YOUR BUILD WITH GRADLE DEVELOCITY

DEMO

LET'S SEE HOW OUR BUILD DOES

TREASUREMAP



X
CACHING

PARALLEL

PROFILES

SCOPES

ANNOTATIONS

ANALYZE

PITFALLS

LIFECYCLES

SETUP MAVEN

PROFILING

DAEMON

LIFECYCLES

CLEAN

DEFAULT

SITE



CLEAN

PRE-CLEAN

CLEAN

POST-CLEAN

(CLEANS THE TARGET FOLDER)



DEFAULT LIFECYCLE

THE PHASES OF BUILD
IN CASE OF PACKAGING
JAR

LIST

PROCES-RESOURCES

(COPY RESOURCES)

COMPILE

(COMPILES SOURCE CODE)

PROCES-TEST-RESOURCES

(COPY RESOURCES)

TEST-COMPILE

(COPY TEST RESOURCES)

TEST

(RUNS UNIT TESTS)

PACKAGE

(CREATES DISTRIBUTABLE ARTIFACT)

INSTALL

(INSTALLS PACKAGE IN LOCAL REPO)

DEPLOY

(COPIES TO REMOTE REPO)

THE PHASES OF SITE

PRE-SITE

SITE

POST-SITE

SITE-DEPLOY

(CREATE DOCUMENTATIONS)

(UPLOAD TO WEBSERVER)



A PHASE IS BIND
TO A PLUGIN AND GOAL

TREASUREMAP

X
CACHING

PARALLEL

PROFILES

SCOPES

ANNOTATIONS

ANALYZE

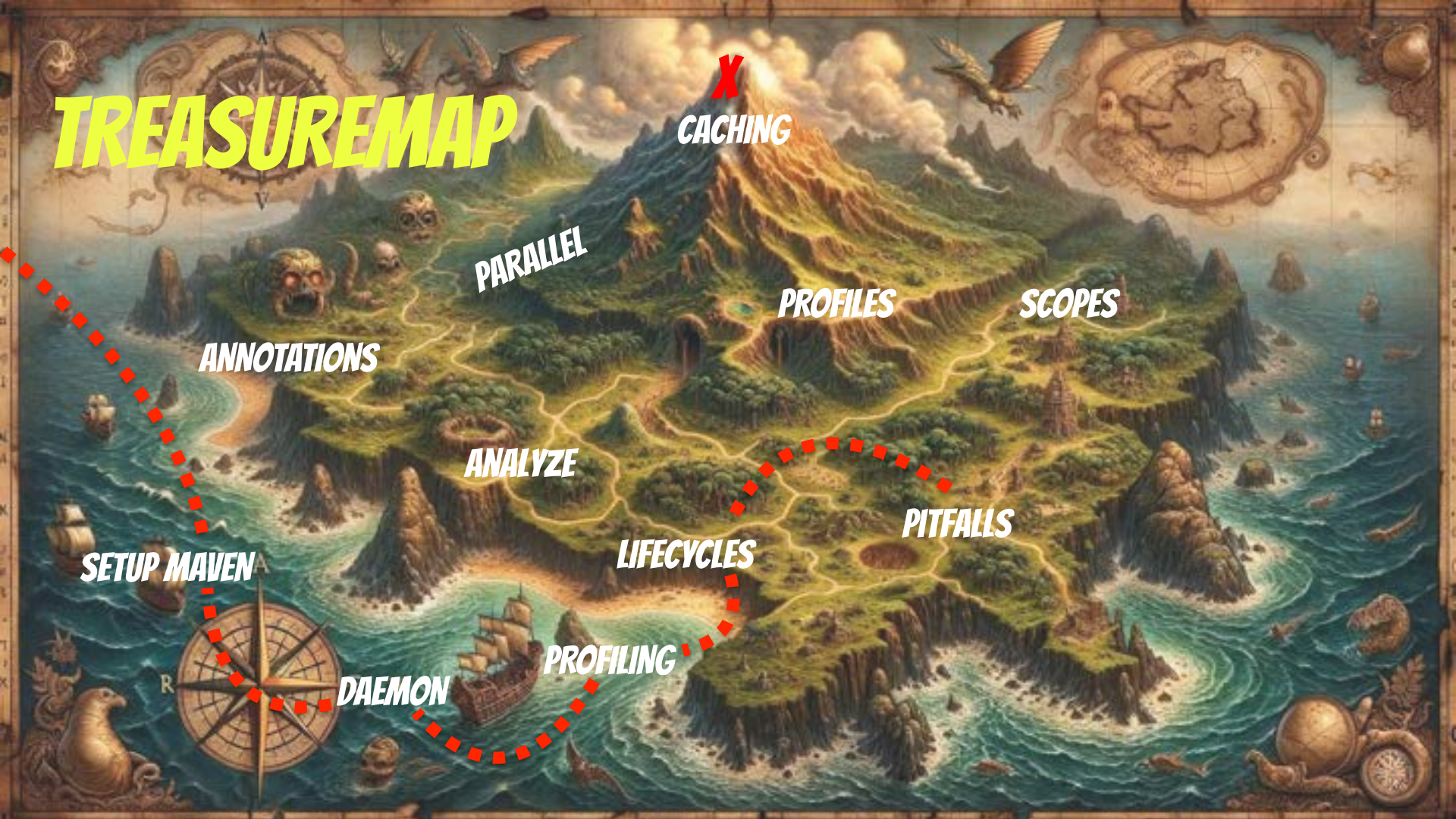
PITFALLS

SETUP MAVEN

LIFECYCLES

PROFILING

DAEMON



THE (MAVEN) PIRATE LIFECYCLES

\$ mvn clean



LIFECYCLES

CLEAN

PHASES

PRE-CLEAN

CLEAN

PLUGIN

ECHO-MAVEN-PLUGIN : PRE-CLEAN

CLEAN

1 *

GOAL

:

CLEAN

ECHO-MAVEN-PLUGIN

DEMO

WHO'S DOING MVN CLEAN INSTALL?



WHAT GIVES PEOPLE FEELINGS OF POWER



WHAT GIVES PEOPLE FEELINGS OF POWER





BY ANDRES ALMIRAY



[HTTPS://GITHUB.COM/AALMIRAY/MVN-CLEAN-INSTALL/](https://github.com/aalmiray/mvn-clean-install/)

THE (MAVEN) PIRATE LIFECYCLES

```
$ mvn clean install
```



LIFECYCLES

PHASES

PLUGIN

1

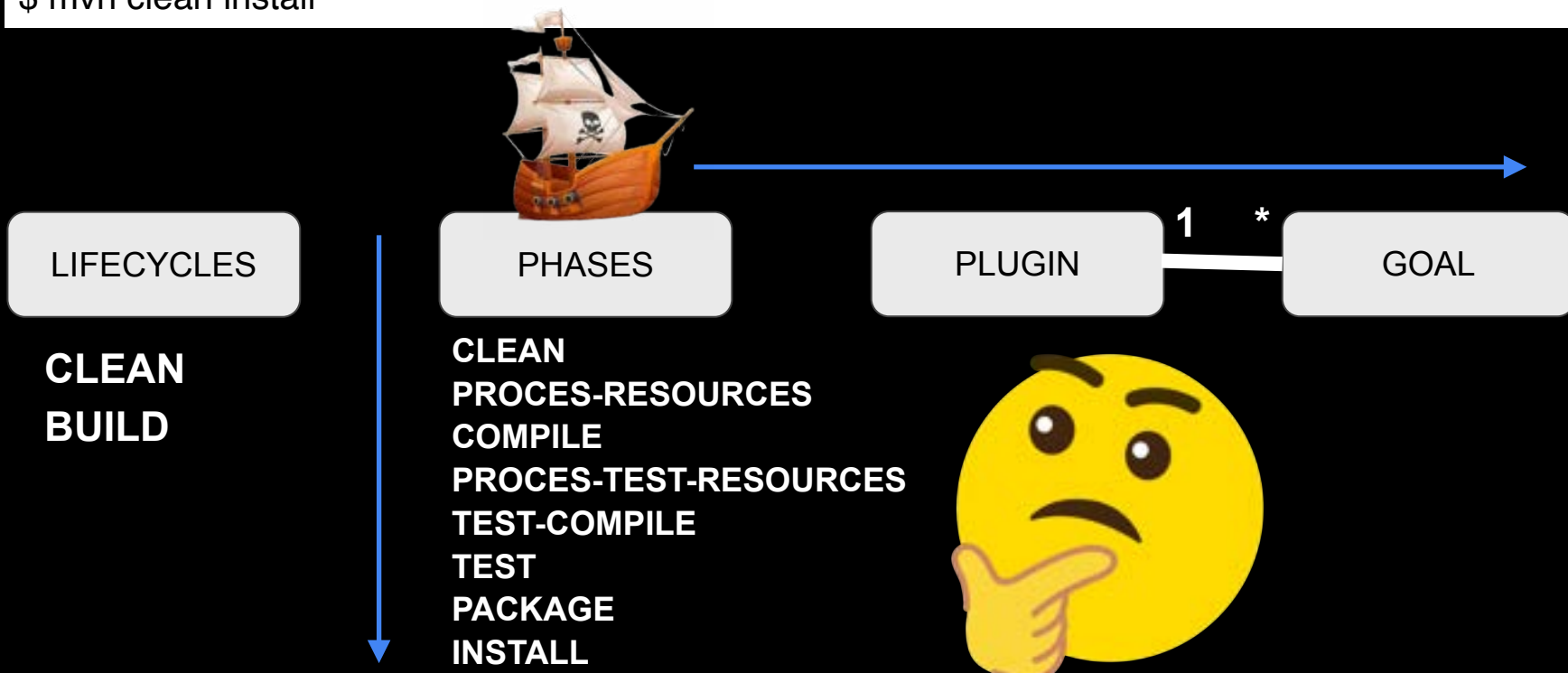
*

GOAL

**CLEAN
BUILD**

THE (MAVEN) PIRATE LIFECYCLES

```
$ mvn clean install
```



THE (MAVEN) PIRATE LIFECYCLES

```
$ mvn clean install
```



LIFECYCLES

**CLEAN
BUILD**

PHASES

**CLEAN
PROCES-RESOURCES
COMPILE
PROCES-TEST-RESOURCES
TEST-COMPILE
TEST
PACKAGE
INSTALL**

PLUGIN

maven-clean-plugin	:	clean
maven-resources-plugin	:	resources
maven-compiler-plugin	:	compile
maven-resources-plugin	:	testResources
maven-compiler-plugin	:	testCompile
maven-surefire-plugin	:	test
maven-jar-plugin	:	jar
maven-install-plugin	:	install

GOAL

MVN TEST

DEMO

THE (MAVEN) PIRATE LIFECYCLES

```
$ mvn site-deploy
```



LIFECYCLES

SITE

PHASES

PLUGIN

1

*

GOAL

THE (MAVEN) PIRATE LIFECYCLES

```
$ mvn site-deploy
```



LIFECYCLES

SITE

PHASES

pre-site
site
post-site
site-deploy

PLUGIN

1

*

GOAL

THE (MAVEN) PIRATE LIFECYCLES

\$ mvn site-deploy



LIFECYCLES

SITE

PHASES

pre-site

site

post-site

site-deploy

PLUGIN

site

site

1

*

GOAL

:

site

:

deploy

BEWARE, YOU ARE NOT INTEGRATION TESTING!

```
<plugin>  
  <groupId>org.apache.maven.plugins</groupId>  
  <artifactId>maven-failsafe-plugin</artifactId>  
  <version>2.22.1</version>  
</plugin>
```


TREASUREMAP

~~X~~
CACHING

PARALLEL

PROFILES

SCOPES

ANNOTATIONS

ANALYZE

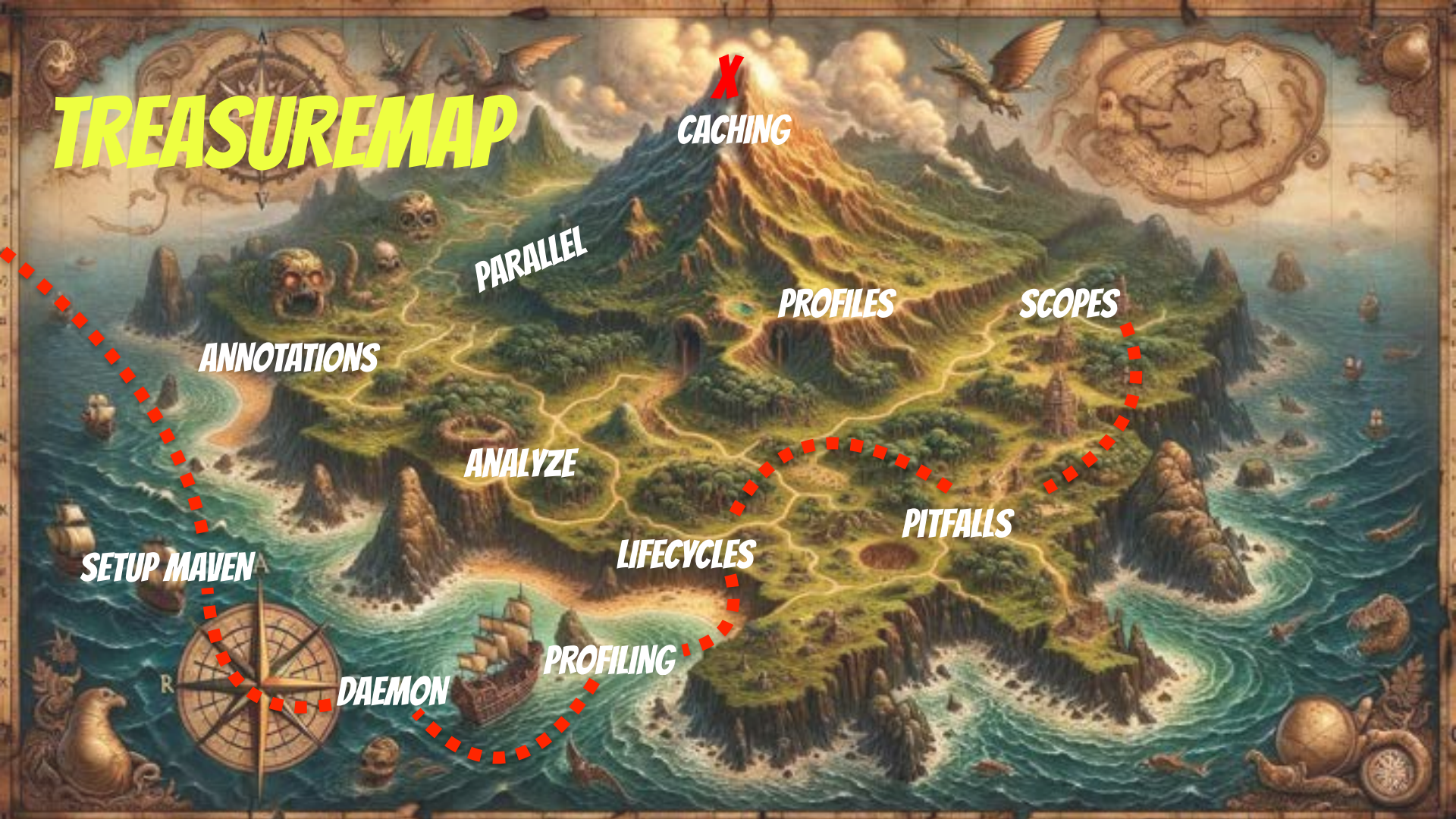
PITFALLS

SETUP MAVEN

LIFECYCLES

PROFILING

DAEMON



***CHANGE YOUR
SCOPES!***

CHANGE YOUR

SCOPES! → DEMO

TREASUREMAP

~~CACHING~~

PARALLEL

PROFILES

SCOPES

ANNOTATIONS

ANALYZE

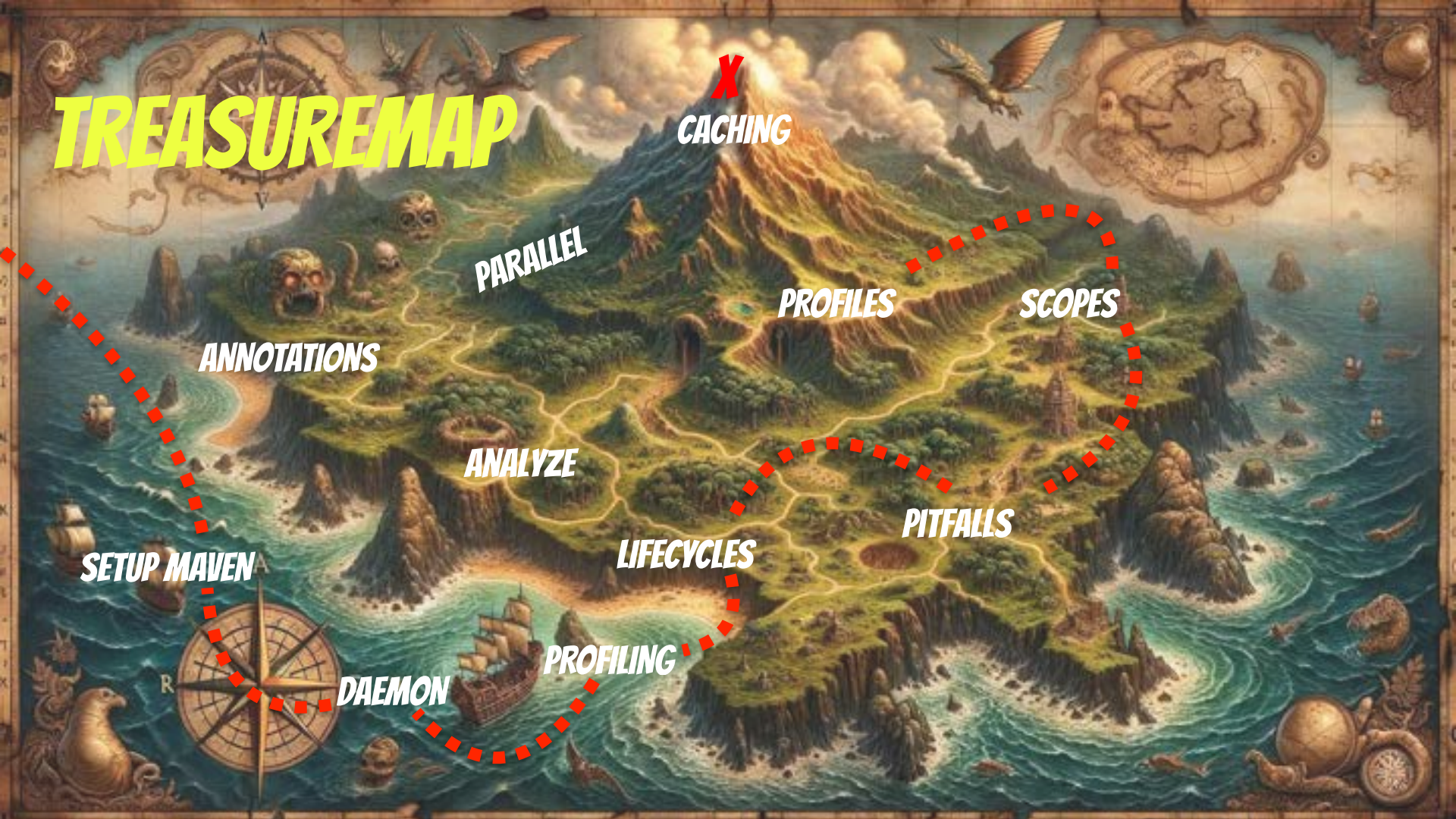
PITFALLS

SETUP MAVEN

LIFECYCLES

PROFILING

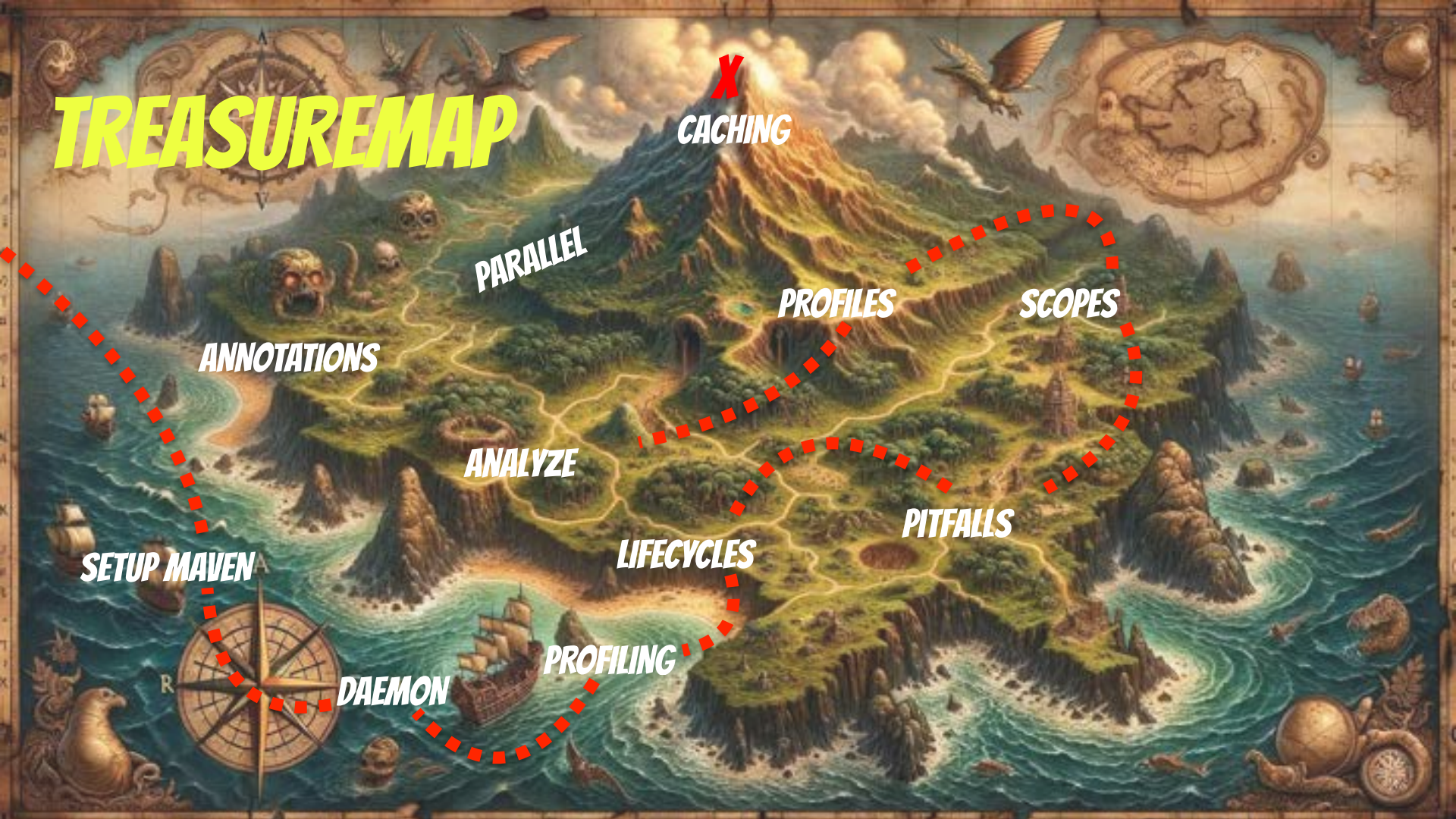
DAEMON



PROFILES

DISCARD OTHER PHASES

TREASUREMAP



X
CACHING

PARALLEL

PROFILES

SCOPES

ANNOTATIONS

ANALYZE

PITFALLS

SETUP MAVEN

LIFECYCLES

PROFILING

DAEMON

DELETING UNUSED DEPENDENCIES

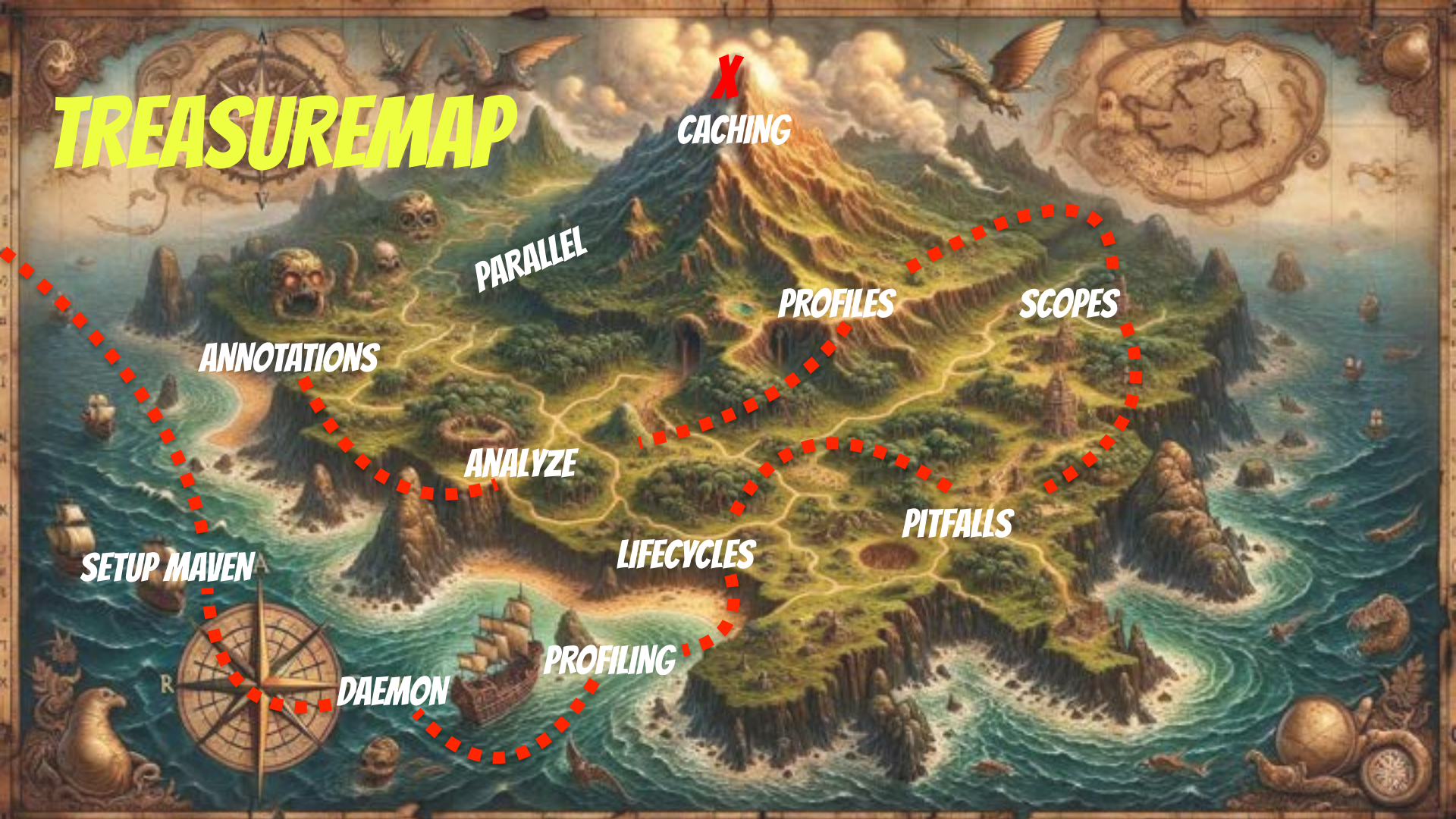
```
$ mvn dependency:analyze
```

OUTPUT:

```
[WARNING] Unused declared dependencies found:  
[WARNING]    org.springframework.boot:spring-boot-starter-actuator  
compile
```

DEMO

TREASUREMAP



X
CACHING

PARALLEL

PROFILES

SCOPES

ANNOTATIONS

ANALYZE

PITFALLS

LIFECYCLES

PROFILING

DAEMON

SETUP MAVEN



TALK WITH

KARL HEINZ MARBAISE

TESTING THE RIGHT SLICES OF YOUR APPLICATION

@SpringBootTest

@WebMvcTest

@JsonTest

@DataJPA Test

@RestClientTests

TESTING THE RIGHT SLICES OF YOUR APPLICATION

WEBMVCTEST

WEBFLUXTEST

{

CONTROLLER

SERVICE

REPOSITORY

Jackson

}

SPRINGBOOTTEST

DATAJPATEST

JDBCTEST

DATAMONGOTEST

DATAREDISTEST

{

JSONTEST



A man in a red polo shirt and blue jeans stands on a stage, pointing towards the presentation screen. To his left is a podium with the 'DEVONX SIUM' logo. The background is a dark space-themed screen with stars.



@SpringBootTest

Test slice annotations

- ✓ @WebMvcTest
- ✓ @RestClientTest
- ✓ @DataJpaTest
- ✓ @GraphQLTest
- ✓ @WebServerServerTest
- ✓ @WebServiceClientTest

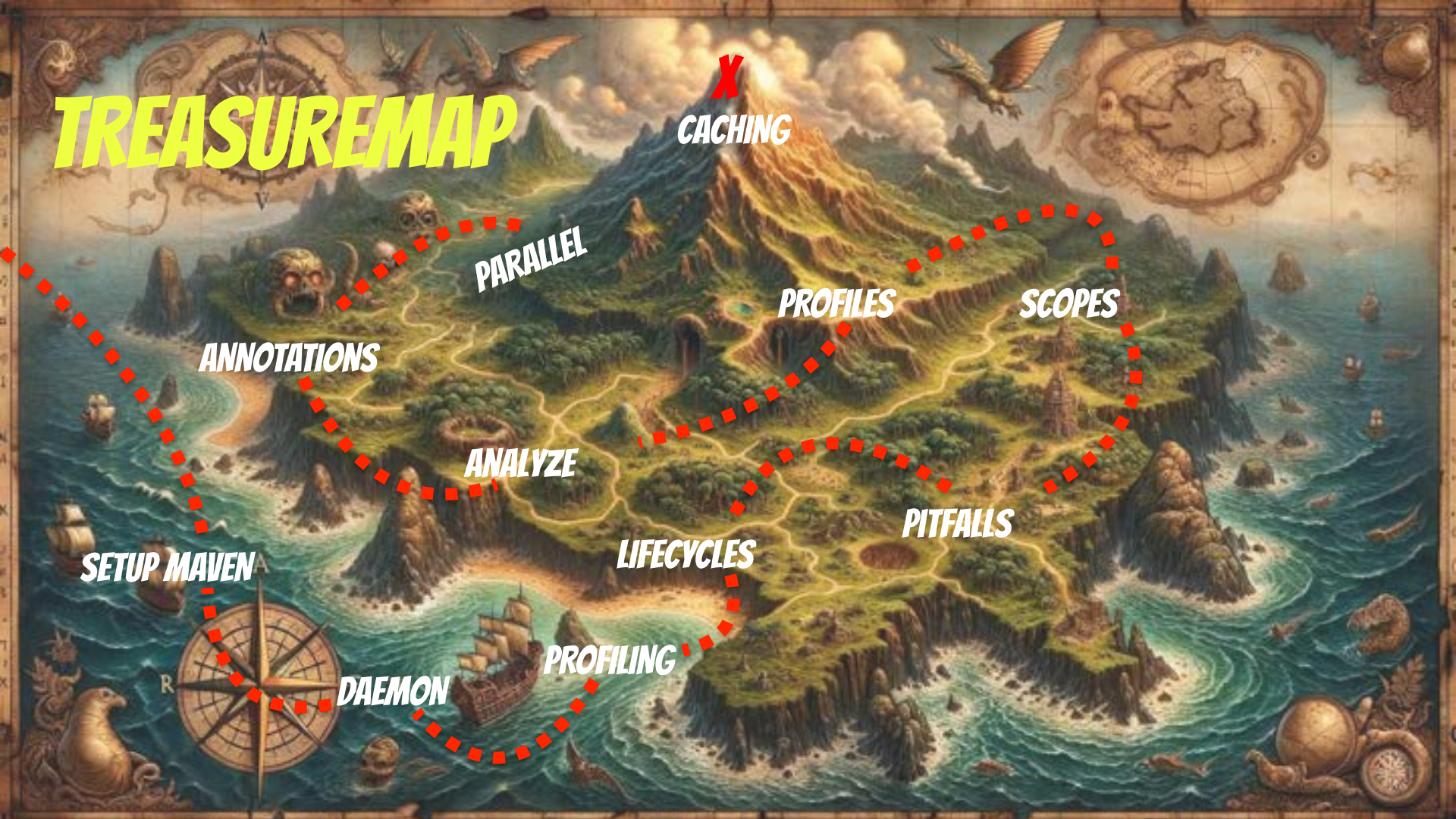
<https://docs.spring.io/spring-boot/docs/current/reference/html/test-auto-configuration.html>
<https://spring.io/blog/2016/08/30/custom-test-slice-with-spring-boot-1.4>

The AWS logo, consisting of a stylized arrow pointing right and the text 'aws'.

The DEVONX logo, with 'DEVONX' in a bold, sans-serif font and a small 'SIUM' logo to its right.

<https://youtu.be/5Z5sa9HpQrE?si=JjIR-DfcEgB7TAg9&t=630>

TREASUREMAP



X
CACHING

PARALLEL

PROFILES

SCOPES

ANNOTATIONS

ANALYZE

PITFALLS

LIFECYCLES

PROFILING

DAEMON

SETUP MAVEN

PARALLEL TESTING

YOUR JUNIT CLASSES

USING MULTIPLE CORES
WHEN BUILDING

USING MULTIPLE THREADS

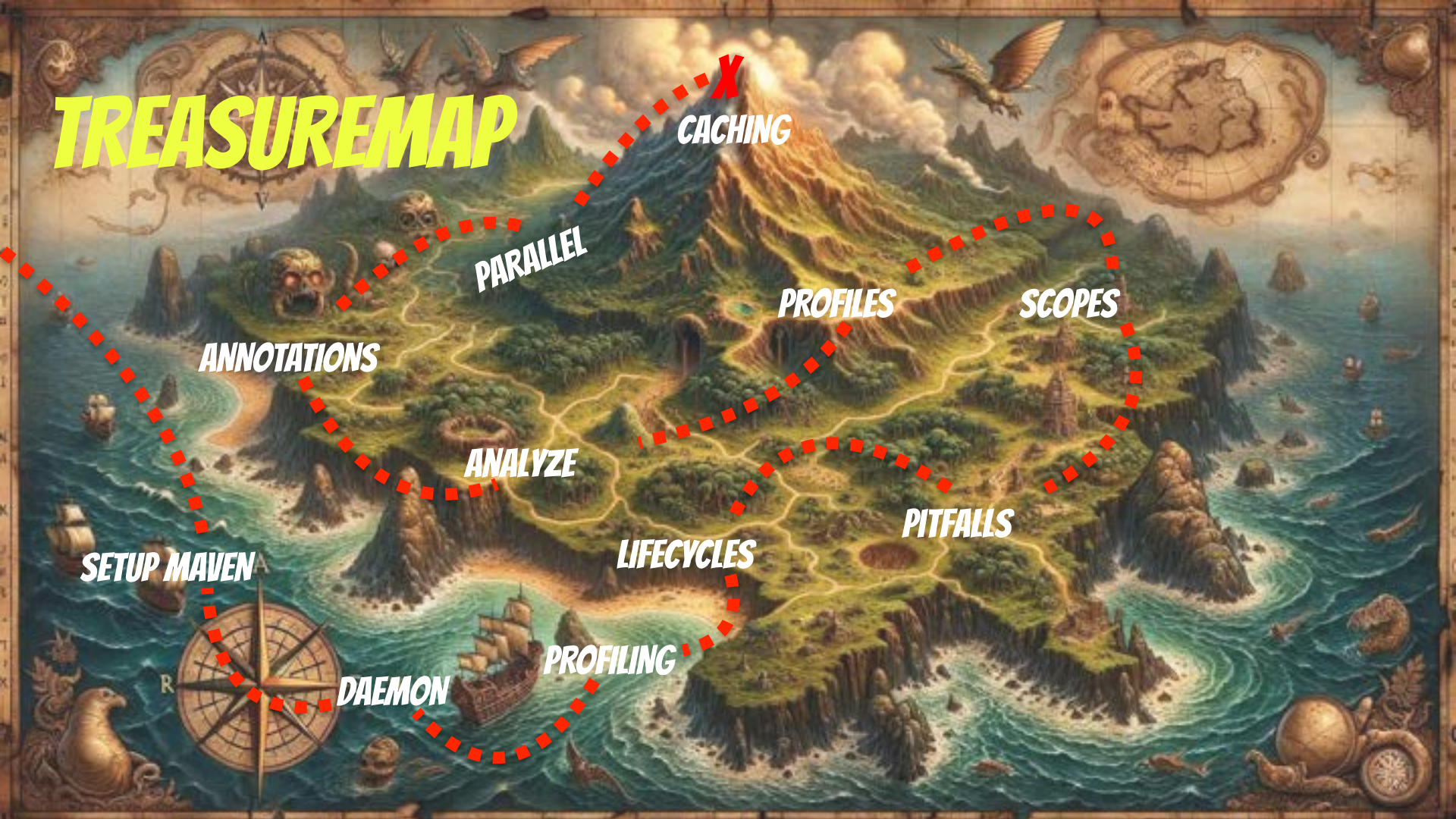
```
$ mvn -T1C clean install
```

MEANS 1 MAVEN BUILD THREAD PER CPU CORE.

IF SOMEONE DIDN'T LOCK THEIR LAPTOP

.MVN/MAVEN.CONFIG

TREASUREMAP

A detailed treasure map illustration. The map shows a large island with a mountain peak, a bay with a ship, and various smaller islands and reefs. The map is decorated with a compass rose in the top left, a skull in the top right, and a compass rose in the bottom left. A red dashed line with diamond markers traces a path across the map, starting from the bottom left, going up to the mountain, then down to the bay, and finally up to the top right. The labels are placed along this path. The word 'TREASUREMAP' is written in large, bold, yellow letters in the top left corner.

CACHING

PARALLEL

PROFILES

SCOPES

ANNOTATIONS

ANALYZE

PITFALLS

SETUP MAVEN

LIFECYCLES

PROFILING

DAEMON

CACHING ARTIFACTS

→ LOCAL

ADJUST YOUR POM FILE

```
<build>
  <extensions>
    <extension>
      <groupId>org.apache.maven.extensions</groupId>
      <artifactId>maven-build-cache-extension</artifactId>
      <version>1.2.2</version>
    </extension>
  </extensions>
</build>
```

OR ADJUST YOUR GRADLE FILE

```
buildCache
{
    local {
        directory = File(rootDir, "build-cache")
        removeUnusedEntriesAfterDays = 30
    }
}
```

WHERE DOES IT SAVE IT CACHE?

Show the files within the build cache directory:

```
$ ls ~/.m2/build-cache/v1
```

com.baeldung

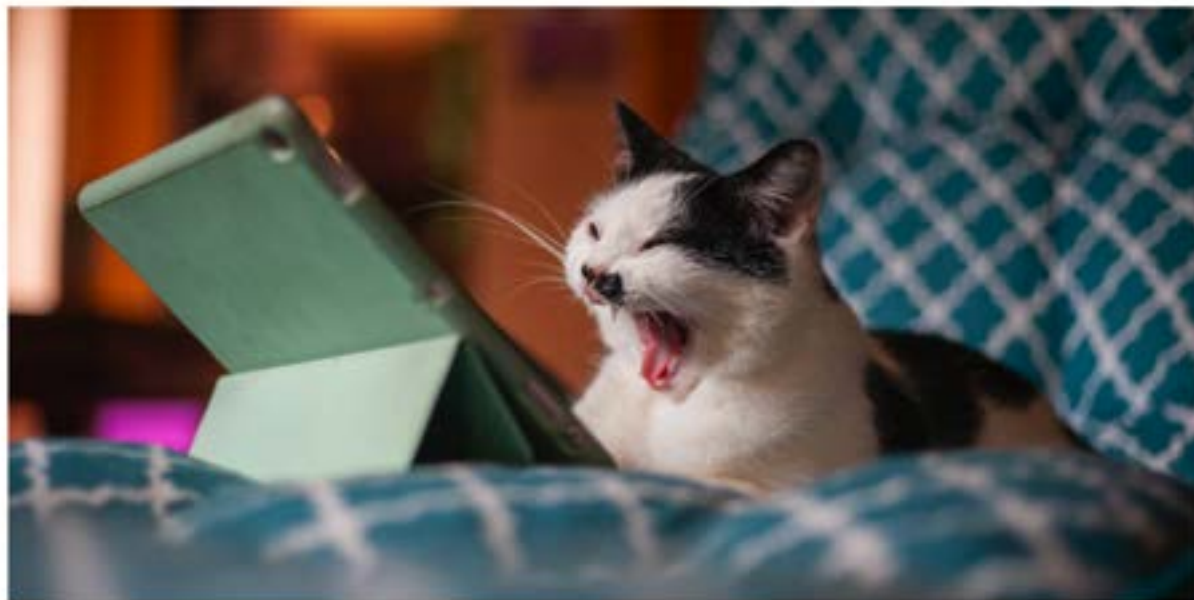
com.baeldung.daomodule

com.baeldung.....

OR ADD IT TO

→ EXTENSIONS.XML

Meow moments, minimize the wait for your build to be completed!



Build caching helps you reduce your local or remote build time! Building a project with Maven or Gradle will save your artifacts to a (remote) cache. The second build will get the artifacts from the cache if nothing changes. It sounds great, right? But why was nobody aware of the feature when I asked about it during our presentation at JFall? Only two people in the room (of 100) knew what it was. This is an excellent opportunity to tell you more about this feature. So say goodbye to your extra coffee breaks, and let's go back to work and code!

CACHING ARTIFACTS

→ REMOTE

GITHUB WORKFLOW

[HTTPS://GITHUB.COM/MARKETPLACE/ACTIONS/MAVEN-CACHE](https://github.com/marketplace/actions/maven-cache)

TREASUREMAP



PARALLEL

PROFILES

SCOPES

ANNOTATIONS

ANALYZE

PITFALLS

SETUP MAVEN

LIFECYCLES

PROFILING

DAEMON



DEMO

DID WE IMPROVED OUR BUILD TIME?

A still life composition featuring a golden chalice, several rolled-up scrolls, and a plaid cloth. The scene is dimly lit, with a warm, golden light highlighting the metallic surfaces of the chalice and the edges of the scrolls. The text "THANK YOU" is overlaid in a bright, bold, yellow font, slanted slightly to the right. The background is dark and out of focus, emphasizing the objects in the foreground.

THANK YOU